Universidad Católica Andrés Bello

Facultad de Ingeniería

Escuela de Ingeniería Informática

Cátedra: Tópicos Especiales De Programación

Proyecto Paradigmas Declarativos

Docente:

- Bello, Janelly.

Integrantes:

- García, Bárbara. CI: 32.340.935.
- Kizer, Elliooth. CI: 31.912.473
- Rivas, Eduardo. CI: 31.384.817

Ciudad Guayana, noviembre de 2026.

Introducción

En la ingeniería de software, la elección del paradigma de programación define la arquitectura, la expresividad y la mantenibilidad de una solución. Mientras que el paradigma imperativo especifica instrucciones secuenciales sobre estados mutables, el paradigma declarativo abstrae el flujo de control para enfocarse en la descripción matemática del problema («qué» resolver en lugar de «cómo» ejecutarlo). Esta abstracción resulta idónea para abordar problemas complejos estructurados sobre grafos ponderados y árboles de búsqueda.

Dentro del enfoque declarativo, la programación lógica y la funcional ofrecen aproximaciones computacionales complementarias. La programación lógica, fundamentada en el cálculo de predicados de primer orden y la ecuación formal de Robert Kowalski, organiza el conocimiento mediante hechos y reglas declarados a través de cláusulas de Horn. En el lenguaje Prolog, un motor de inferencia resuelve las consultas aplicando unificación de términos y resolución SLD con backtracking implícito. Por su parte, la programación funcional se basa en el cálculo lambda y la evaluación de funciones puras desprovistas de efectos secundarios. Bajo el lenguaje Haskell, este modelo garantiza transparencia referencial, inmutabilidad de datos y un sistema de tipado estático con inferencia.

El presente proyecto abarca el desarrollo del motor analítico para el sistema de itinerarios interurbanos de la empresa LogiExpress. La problemática exige procesar dinámicamente un conjunto de vuelos en formato militar (HH:MM) para determinar rutas válidas entre diversas ciudades de Venezuela. El sistema evalúa restricciones de operación por días, un Tiempo Mínimo de Conexión (TMC) global entre vuelos consecutivos, prevención de ciclos urbanos y límites en el número de escalas. Asimismo, busca optimizar los itinerarios bajo un doble criterio: la minimización del costo económico del boleto y la reducción del tiempo total de viaje.

La trascendencia del trabajo reside en contrastar ambos paradigmas frente a un mismo problema de ingeniería. Mientras Prolog explora las rutas mediante backtracking sintáctico, Haskell resuelve la búsqueda en profundidad (DFS) de forma inmutable usando funciones de alto orden (map, filter, fold), list comprehensions y evaluación perezosa. El informe documenta la representación abstracta de datos, los predicados y funciones investigados, el desglose de responsabilidades del equipo, el flujo de trabajo con control de versiones en Git, las evidencias de ejecución y la comparativa técnica final.

Explicación del Modelado de Datos

La red de transporte se abstrae como un grafo dirigido ponderado: los nodos son ciudades/aeropuertos y las aristas son vuelos con atributos multidimensionales (salida, llegada, precio, código y frecuencia). Ambos paradigmas normalizan las horas a minutos absolutos desde la medianoche (p. ej. 07:00 → 420), lo que reduce las restricciones de tiempo a la aritmética entera.

Paradigma lógico (Prolog).

La base de conocimiento se construye dinámicamente mediante el predicado vuelo/7, un hecho por cada línea de vuelos.txt. El TMC se gestiona como un hecho dinámico independiente, reconfigurable en ejecución con retractall/1 y assertz/1.

```
:- dynamic vuelo/7.
:- dynamic tmc_config/1.

% vuelo(Origen, Destino, SalidaMin, LlegadaMin, Precio, IDVuelo, ListaDias)
vuelo('Pto Ordaz', 'Maiquetia', 420, 480, 5600, 'Vlo#1', ['Diario']).
vuelo('Maiquetia', 'Porlamar', 540, 600, 4800, 'Vlo#2', ['L','M','Mi','J','V']).

tmc_config(45).  % Tiempo Mínimo de Conexión (minutos)
```

Las rutas no se almacenan: se construyen durante la búsqueda como listas de términos vuelo/7 (estructura [Cabeza|Cola]), guiadas por el backtracking.

Paradigma funcional (Haskell).

El modelo es explícito, inmutable y fuertemente tipado, definido en Tipos.hs mediante **tipos algebraicos de datos (ADT)** y record syntax:

```
type Ciudad = String; type Minutos = Int; type Precio = Int
type CodigoVuelo = String; type TMC = Int

data Dia = L | M | Mi | J | V | S | D | Diario deriving (Show, Eq, Read)
data Vuelo = Vuelo
  { origen :: Ciudad, destino :: Ciudad
  , salida :: Minutos, llegada :: Minutos
  , precio :: Precio, codigoVuelo :: CodigoVuelo
  , dias :: [Dia] } deriving (Show, Eq)
type Grafo = [Vuelo]  -- el grafo es una lista inmutable de vuelos
type Ruta  = [Vuelo]  -- una ruta es una secuencia ordenada de vuelos
```

El DFS no requiere una estructura de nodos y aristas explícita: filtra los vuelos cuyo origen coincide con la ciudad actual. El grafo y el TMC viajan como parámetros inmutables entre funciones puras.

Funciones y Predicados Investigados

El desarrollo del motor analítico de LogiExpress requirió una investigación profunda sobre los mecanismos de Entrada/Salida (I/O), manipulación de datos, estructuras recursivas y meta-programación en ambos paradigmas. A continuación se desglosan las funciones y predicados desarrollados e investigados, organizados por módulo y por su respectivo responsable técnico.

Módulo 1: Carga de Datos, Parseo e Interfaz CLI.

Responsable: Elliooth Kizer

El Módulo 1 constituye la capa de persistencia y comunicación con el usuario. Su objetivo es leer la base de datos plana vuelos.txt, normalizar los horarios militares (HH:MM) a minutos absolutos del día y proveer un menú interactivo en consola.

Paradigma Lógico (Prolog - parser.pl y main.pl)

En Prolog, la base de datos no es una estructura estática recibida por argumento, sino un conjunto de cláusulas asentadas dinámicamente en el espacio de trabajo del intérprete (Knowledge Base).

- **:- dynamic vuelo/7, tmc_config/1.** Declaración que informa al compilador de SWI-Prolog que los predicados de vuelo y la configuración global de TMC pueden ser modificados dinámicamente en tiempo de ejecución (runtime).
- **assertz/1 y retractall/1.** Utilizados para gestionar el estado dinámico. retractall(vuelo(_,_,_,_,_,_,_)) purga hechos previos para evitar duplicaciones antes de invocar assertz(vuelo(...)), el cual inserta nuevos hechos al final de la base de conocimientos.
- **open/3, read_line_to_string/2 y close/1.** Primitivas de I/O para la apertura del archivo de texto en modo lectura, la extracción línea por línea en formato de cadena y la liberación de descriptores de archivos.
- **split_string/4 y atom_number/2.** Empleados para segmentar cada registro delimitado por '|' y convertir las cadenas numéricas (precios y componentes de horas) en términos de tipo entero (integer).
- **hora_a_minutos/2.** Predicado auxiliar que descompone una cadena "HH:MM" utilizando split_string/4 y calcula los minutos absolutos desde medianoche: Total = (H × 60) + M, donde H son las horas y M los minutos de la cadena.

Paradigma Funcional (Haskell - Parser.hs y Main.hs)

En Haskell, el procesamiento de I/O se encuentra estrictamente aislado dentro de la mónada IO, preservando la pureza del resto del sistema.

- **readFile :: FilePath -> IO String.** Función de I/O con evaluación perezosa (lazy) que expone el contenido del archivo vuelos.txt como una cadena inmutable, leída bajo demanda a medida que se consume.
- **lines :: String -> [String].** Función pura de la biblioteca estándar que descompone la cadena de texto en una lista de líneas individuales.
- **parsearLinea :: String -> Vuelo.** Función pura que recibe una línea delimitada por '|', extrae sus componentes mediante splitOn o combinadores de listas, y retorna una instancia tipada del registro Vuelo.
- **horaAMinutos :: String -> Minutos.** Convierte la representación en formato militar "HH:MM" a un entero Int mediante coincidencia de patrones (pattern matching) y operaciones aritméticas puras.
- **parsearDias :: String -> [Dia].** Mapea la representación en texto de las frecuencias (ej. "L,M,V" o "Diario") a una lista de constructores del tipo algebraico suma Dia.

- **Bucle CLI (do blocks y putStrLn/getLine).** Interfaz de línea de comandos estructurada secuencialmente mediante notación do sobre la mónada IO, capturando la entrada del usuario y pasando el estado del Grafo = [Vuelo] a las funciones puras de los módulos subsiguientes.

Módulo 2: Motor de Búsqueda DFS y Validación de Restricciones.

Responsable: Bárbara García

El Módulo 2 representa el núcleo algorítmico del proyecto. Implementa la búsqueda en profundidad (Depth-First Search, DFS) para explorar exhaustivamente las rutas posibles entre dos ciudades, evaluando simultáneamente cuatro restricciones operacionales: compatibilidad de días, tiempo mínimo de conexión (TMC) en cada escala, control estricto de aciclicidad y límite máximo de escalas (K). La búsqueda se realiza para un día de operación específico y los horarios se comparan como minutos desde la medianoche de ese día.

Paradigma Lógico (Prolog - rutas.pl)

El motor lógico aprovecha la resolución SLD y el backtracking implícito de Prolog para recorrer el árbol de estados.

- **buscar_rutas(Origen, Destino, Dia, MaxEscalas, Ruta).** Predicado de entrada que inicializa la búsqueda en profundidad invocando el motor recursivo buscar_ruta_dfs/8 con una lista de ciudades visitadas [Origen] para controlar la aciclicidad. El TMC se obtiene internamente mediante obtener_tmc/1.
- **buscar_ruta_dfs/8 (caso base y caso recursivo).**
  - ***Caso base:*** satisface la consulta cuando la ciudad de llegada del vuelo actual coincide con el Destino solicitado.
  - ***Caso recursivo:*** realiza la unificación con un hecho vuelo(Origen, Intermedio, ...) válido, verifica que Intermedio no haya sido visitado previamente mediante la negación por fallo \+ member(Intermedio, Visitadas), decrementa el contador de escalas disponibles (K − 1) y realiza la llamada recursiva acumulando la ruta.
- **\+ member/2.** Garantiza la aciclicidad impidiendo que el algoritmo visite dos veces la misma ciudad dentro de un mismo itinerario.
- **cumple_tmc/3.** Predicado de control que recibe la hora de llegada del vuelo previo (V1), la hora de salida del vuelo actual (V2) y el TMC vigente, exigiendo formalmente: Salida(V2) ≥ Llegada(V1) + TMC.
- **dia_compatible/2.** Comprueba mediante unificación si el día de viaje solicitado está contenido en la lista de días de operación del vuelo o si el vuelo opera con la etiqueta 'Diario'.

Paradigma Funcional (Haskell - Rutas.hs)

En Haskell, al no existir backtracking nativo en el lenguaje, la búsqueda DFS se implementa mediante recursión explícita sobre listas y comprensiones de listas (list comprehensions).

- **buscarRutas :: Grafo -> Ciudad -> Ciudad -> Dia -> Maybe Int -> TMC -> [Ruta].** Función pura de punto de entrada que desencadena el algoritmo DFS pasando el grafo inmutable.
- **dfs :: Grafo -> Ciudad -> Ciudad -> Dia -> Int -> TMC -> [Ciudad] -> Ruta -> [Ruta].** Genera recursivamente todas las ramas factibles. Utiliza evaluación perezosa para construir la lista de todas las rutas sin agotar la memoria antes de ser consumidas. El límite de escalas K se controla con el parámetro entero, que se decrementa en cada llamada recursiva.
- **notElem :: Eq a => a -> [a] -> Bool.** Función de la biblioteca estándar que valida la aciclicidad verificando que la siguiente ciudad destino no se encuentre presente en la lista de ciudades acumuladas [Ciudad].

- **Comprensiones de listas y guardas (|).** Sintaxis pura que integra en un solo bloque conciso el filtrado de vuelos desde la ciudad actual, la validación de días, el control de aciclicidad y la verificación de la conexión TMC:

```
[ vuelo : rutaRestante | vuelo <- grafo
                       , origen vuelo == ciudadActual
                       , esDiaCompatible dia (dias vuelo)
                       , notElem (destino vuelo) visitadas
                       , validarTMC ultimoVuelo vuelo tmc
                       , rutaRestante <- dfs grafo (destino vuelo) destinoFinal ... ]
```

Módulo 3: Optimización, Métricas y Ordenamiento por Quicksort.

Responsable: Eduardo Rivas

El Módulo 3 se encarga de cuantificar el costo total (tarifa en $) y el tiempo total de viaje (duración de los vuelos más las esperas en escala), ordenar las soluciones obtenidas mediante el algoritmo Quicksort puro y presentar las mejores opciones (la ruta más económica y la ruta más rápida).

Paradigma Lógico (Prolog - optimizacion.pl)

- **findall/3.** Meta-predicado fundamental de recopilación de soluciones. Evalúa la consulta de búsqueda de rutas por backtracking y consolida todos los itinerarios válidos en una lista de Prolog para su posterior ordenamiento: findall(Ruta, buscar_rutas(Origen, Destino, Dia, MaxEscalas, Ruta), ListaDeRutas).
- **costo_total_ruta/2.** Recorre de forma recursiva la lista de hechos vuelo/7 sumando el argumento posicional relativo al precio.
- **tiempo_total_viaje/2.** Recorre recursivamente los vuelos de un itinerario, sumando la duración propia de cada vuelo (Llegada − Salida) y el tiempo de espera en conexión entre vuelos consecutivos V1 y V2.
- **quicksort/3 y call/N.** Se desarrolló un Quicksort de alto orden recursivo puro. Utiliza el meta-predicado call/N para aplicar dinámicamente el criterio de comparación recibido como argumento (comparar_costo o comparar_tiempo), particionando la lista en sublistas de elementos menores y mayores respecto al pivote.

Paradigma Funcional (Haskell - Optimizacion.hs)

- **costoTotalRuta :: Ruta -> Precio.** Suma los precios del billete mediante comprensiones de listas (o, alternativamente, la reducción foldl) sobre los vuelos del itinerario:
  `costoTotalRuta ruta = sum [ precio v | v <- ruta ]`
- **tiempoTotalViaje :: Ruta -> Minutos.** Calcula la diferencia entre la llegada del último segmento y la salida del primero: llegada (last ruta) - salida (head ruta).
- **quicksortPor :: Ord b => (a -> b) -> [a] -> [a].** Quicksort declarativo basado en funciones de alto orden y comprensiones de listas:

```
quicksortPor _ [] = []
quicksortPor cmp (p:xs) =
   quicksortPor cmp [x | x <- xs, cmp x p == LT || cmp x p == EQ] ++ [p] ++
   quicksortPor cmp [x | x <- xs, cmp x p == GT]
```

- **mejorRutaPor con foldl / selección de óptimos.** Extrae de la lista de rutas la de mínimo valor para las métricas de costo y tiempo recorriéndola una sola vez, sin necesidad de ordenarla. Para el reporte ordenado se usa quicksortPor y, gracias a la evaluación perezosa, ordena usando una función de clave (key function), no una función de comparación.

Resumen de actividades y responsables

1. Elliooth Kizer: Módulo 1 (Persistencia, Parseo e Interfaz CLI)

Área de dominio: capa de Entrada/Salida (I/O), parseo de datos y menús interactivos por consola.

Actividades desarrolladas:

- Diseño y construcción del módulo de lectura del archivo plano vuelos.txt delimitado por el carácter '|'.
- Implementación de la rutina de conversión de formatos de tiempo militar (HH:MM) a enteros que representan minutos absolutos contados desde medianoche.
- En Prolog **(parser.pl y main.pl):** definición de los hechos dinámicos vuelo/7 y tmc_config/1, y gestión del estado mediante assertz/1 y retractall/1.
- En Haskell **(Tipos.hs, Parser.hs y Main.hs):** definición del sistema de tipos algebraicos (data Dia, data Vuelo), tipos sinónimos (type Grafo = [Vuelo]) y aislamiento de operaciones I/O en la mónada IO.
- Construcción de los menús CLI interactivos en consola en ambos lenguajes.
- **Trabajo en conjunto:** redacción de la explicación teórica del Módulo 1, definición del modelado de datos y revisión técnica del informe.

2. Bárbara García: Módulo 2 (Motor de Búsqueda DFS y Restricciones)

Área de dominio: núcleo de exploración de grafos y validación de reglas de negocio operacionales.

Actividades desarrolladas:

- Diseño del algoritmo de búsqueda en profundidad (Depth-First Search, DFS) sobre la estructura de grafo interurbano.
- Implementación de la validación simultánea de las cuatro restricciones del problema: compatibilidad de días operacionales, tiempo mínimo de conexión (TMC), control estricto de aciclicidad urbana y delimitación del tope máximo de escalas (K).
- En Prolog **(rutas.pl):** motor de inferencia recursivo buscar_ruta_dfs/8 apoyado en el backtracking implícito de la resolución SLD y prevención de ciclos mediante negación por fallo \+ member/2.
- En Haskell **(Rutas.hs):** búsqueda DFS declarativa pura mediante comprensiones de listas (list comprehensions), guardas y filtrado inmutable con notElem.
- **Trabajo en conjunto:** redacción de la sección del motor DFS, validación cruzada de resultados e integración de las evidencias de ejecución, y estructuración del Informe Técnico.

3. Eduardo Rivas: Módulo 3 (Optimización y Métricas)

Área de dominio: algoritmos de ordenamiento y cálculo de métricas de rendimiento.

Actividades desarrolladas:

- Desarrollo de las funciones y predicados de cálculo de métricas de itinerario: costo total del billete en dólares ($) y tiempo total de viaje (duración de los vuelos más los tiempos de espera en escala).
- Investigación e implementación del algoritmo Quicksort puro recursivo en ambos paradigmas, sin librerías externas.
- En Prolog **(optimizacion.pl):** recopilación exhaustiva de soluciones con findall/3 y Quicksort de alto orden utilizando meta-programación con call/N.
- En Haskell **(Optimizacion.hs):** Quicksort declarativo con comprensiones de listas (quicksortPor) y selección de óptimos aprovechando la evaluación perezosa.
- **Trabajo en conjunto:** redacción de la explicación teórica del Módulo 3, consolidación del cuadro comparativo final y estructuración del Informe Técnico Comparativo.

Flujo de Trabajo Colaborativo y Control de Versiones.

El desarrollo del proyecto se gestionó con Git, usando un repositorio remoto compartido por los tres integrantes. El flujo de trabajo adoptó las siguientes prácticas de ingeniería de software:

1. **Modularidad de archivos.** Se mantuvo una separación estricta entre las carpetas /prolog y /haskell, evitando interferencias en la compilación y permitiendo pruebas independientes de cada módulo.
2. **Integración incremental de módulos.** Cada integrante trabajó los predicados de Prolog y las funciones de Haskell correspondientes a su módulo, asegurando que las firmas de las funciones respetaran los tipos de datos abstractos acordados en el Módulo 1.
3. **Documentación conjunta.** La documentación técnica se redactó de forma colaborativa entre los tres integrantes, realizando revisiones por pares para asegurar la equivalencia formal en la explicación de ambos paradigmas y la exactitud de los resultados numéricos presentados.

Evidencias de Ejecución por Consola

Ambas implementaciones comparten el mismo menú de 9 opciones y se ejecutaron sobre los 20 vuelos de vuelos.txt con TMC = 45 min (SWI-Prolog 10.0.2 y GHC 9.x). A continuación se presentan las capturas del flujo completo en Prolog y el menú principal de Haskell. Los montos se imprimen con el símbolo $.

1. Menú Principal Paradigma Lógico - Prolog.

```
[OK] Base de datos cargada exitosamente: 20 vuelos registrados.

------------------------------------------------------------------------
           LogiExpress - Sistema analitico de vuelos
                    (Paradigma logico)
------------------------------------------------------------------------
 [TMC actual: 45 min]
------------------------------------------------------------------------
  1. Cargar base de datos (vuelos.txt)
  2. Configurar tiempo minimo de conexion (TMC)
  3. Consultar vuelos que salen de una ciudad
  4. Consultar vuelos que llegan a una ciudad
  5. Buscar rutas validas
  6. Buscar ruta de menor costo
  7. Buscar ruta de menor tiempo
  8. Ver rutas ordenadas por costo o tiempo
  9. Salir
------------------------------------------------------------------------
Seleccione una opcion [1-9]:
```

2. Carga Base de datos.

```
1

--- Carga de base de datos ---
Ingrese la ruta del archivo [Enter para usar vuelos.txt por defecto]:
[OK] Base de datos cargada exitosamente: 20 vuelos registrados.

Presione Enter para continuar...
```

3. Configurar el Tiempo mínimo de conexión TMC.

```
Seleccione una opcion [1-9]: 2

--- Configurar tiempo minimo de conexion (TMC) ---
TMC actual: 45 minutos.
Ingrese nuevo TMC en minutos (ej. 45): 45
[Exito] TMC actualizado a 45 minutos.

Presione Enter para continuar...
```

4. Vuelos que salen de una ciudad

```
3

--- Vuelos que salen de una ciudad ---
Ingrese nombre de la ciudad de origen (ej. Maiquetia, Pto Ordaz): Maiquetia
Total de vuelos que salen de "Maiquetia": 5
-------------------------------------------------------------------------------
 Codigo |      Origen       |     Destino       | Salida | Llegada |  Precio | Dias
-------------------------------------------------------------------------------
 Vlo#2  | Maiquetia         | Porlamar          | 09:00  | 10:00   | $4800   | L-M-Mi-J-V
 Vlo#3  | Maiquetia         | Pto Ordaz         | 15:00  | 16:00   | $5300   | L-M-V
 Vlo#10 | Maiquetia         | Barquisimeto      | 06:30  | 07:30   | $4200   | Diario
 Vlo#14 | Maiquetia         | Barcelona         | 11:00  | 11:50   | $4100   | Diario
 Vlo#17 | Maiquetia         | San Antonio       | 07:00  | 08:30   | $8500   | L-Mi-V
-------------------------------------------------------------------------------

Presione Enter para continuar...
```

5. Vuelos que llegan a una ciudad

```
Seleccione una opcion [1-9]: 4

--- Vuelos que llegan a una ciudad ---
Ingrese nombre de la ciudad de destino (ej. Porlamar, Valencia): Porlamar
Total de vuelos que llegan a "Porlamar": 3
-------------------------------------------------------------------------------
 Codigo |      Origen       |     Destino       | Salida | Llegada |  Precio | Dias
-------------------------------------------------------------------------------
 Vlo#2  | Maiquetia         | Porlamar          | 09:00  | 10:00   | $4800   | L-M-Mi-J-V
 Vlo#9  | Valencia          | Porlamar          | 17:00  | 18:00   | $4300   | L-S-D
 Vlo#15 | Barcelona         | Porlamar          | 13:00  | 13:40   | $3500   | Diario
-------------------------------------------------------------------------------

Presione Enter para continuar...
```

6. Buscar Rutas Validas

```
Seleccione una opcion [1-9]: 5

--- Buscar rutas validas ---
Ciudad de Origen: Barquisimeto
Ciudad de Destino: Maiquetia
Dia de la semana (ej. Lunes, Martes o L, M, Mi, J, V, S, D): L
Maximo numero de escalas [Presione Enter para sin limite]:
Se encontraron 2 rutas validas de "Barquisimeto" a "Maiquetia" para el dia L:
[Ruta #1] Escalas: 1 | Costo Total: $9400 | Tiempo Total: 570 min (9h 30m)
  Segmentos:
    (1) [Vlo#11] Barquisimeto -> Maracaibo | 08:30 - 09:30 (60 min) | $3900
        * Conexion en Maracaibo: espera de 450 minutos.
    (2) [Vlo#5] Maracaibo -> Maiquetia | 17:00 - 18:00 (60 min) | $5500
------------------------------------------------------------------------
[Ruta #2] Escalas: 0 | Costo Total: $4500 | Tiempo Total: 60 min (1h 0m)
  Segmentos:
    (1) [Vlo#12] Barquisimeto -> Maiquetia | 18:00 - 19:00 (60 min) | $4500
------------------------------------------------------------------------

Presione Enter para continuar...
```

7. Ruta de menos costo

```
Seleccione una opcion [1-9]: 6

--- Ruta de menor costo ---
Ciudad de Origen: Barquisimeto
Ciudad de Destino: Maiquetia
Dia de la semana (ej. Lunes, Martes o L, M, Mi, J, V, S, D): L
Maximo numero de escalas [Presione Enter para sin limite]:
------------------------------------------------------------------------
>>> Ruta de menor costo (Total: $4500) <<<
------------------------------------------------------------------------
[Ruta #1] Escalas: 0 | Costo Total: $4500 | Tiempo Total: 60 min (1h 0m)
  Segmentos:
    (1) [Vlo#12] Barquisimeto -> Maiquetia | 18:00 - 19:00 (60 min) | $4500
------------------------------------------------------------------------

Presione Enter para continuar...
```

8. Ruta de menor Tiempo

```
Seleccione una opcion [1-9]: 7

--- Ruta de menor tiempo ---
Ciudad de Origen: Barquisimeto
Ciudad de Destino: Maiquetia
Dia de la semana (ej. Lunes, Martes o L, M, Mi, J, V, S, D): L
Maximo numero de escalas [Presione Enter para sin limite]:
------------------------------------------------------------------------
>>> Ruta de menor tiempo (Duracion: 60 min [1h 0m]) <<<
------------------------------------------------------------------------
[Ruta #1] Escalas: 0 | Costo Total: $4500 | Tiempo Total: 60 min (1h 0m)
  Segmentos:
    (1) [Vlo#12] Barquisimeto -> Maiquetia | 18:00 - 19:00 (60 min) | $4500
------------------------------------------------------------------------

Presione Enter para continuar...
```

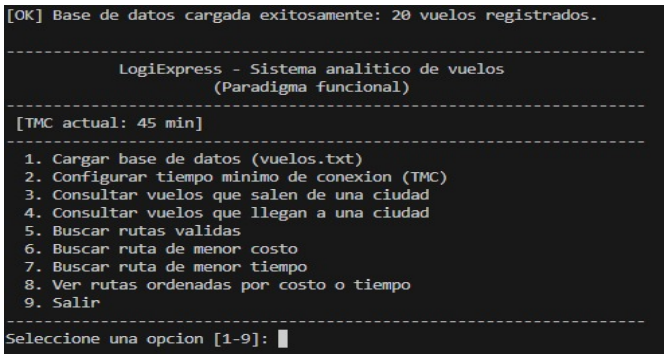
9. Rutas Ordenadas

```
Seleccione una opcion [1-9]: 8

--- Reporte ordenado de rutas ---
Ciudad de Origen: Barquisimeto
Ciudad de Destino: Maiquetia
Dia de la semana (ej. Lunes, Martes o L, M, Mi, J, V, S, D): L
Maximo numero de escalas [Presione Enter para sin limite]:
Seleccione el criterio de ordenamiento:
  1. Ordenar por menor costo
  2. Ordenar por menor tiempo
Criterio [1-2]: 1
>>> Rutas ordenadas por menor costo: <<<
[Ruta #1] Escalas: 0 | Costo Total: $4500 | Tiempo Total: 60 min (1h 0m)
  Segmentos:
    (1) [Vlo#12] Barquisimeto -> Maiquetia | 18:00 - 19:00 (60 min) | $4500
------------------------------------------------------------------------
[Ruta #2] Escalas: 1 | Costo Total: $9400 | Tiempo Total: 570 min (9h 30m)
  Segmentos:
    (1) [Vlo#11] Barquisimeto -> Maracaibo | 08:30 - 09:30 (60 min) | $3900
        * Conexion en Maracaibo: espera de 450 minutos.
    (2) [Vlo#5] Maracaibo -> Maiquetia | 17:00 - 18:00 (60 min) | $5500
------------------------------------------------------------------------

Presione Enter para continuar...
```

10. Menú Principal Paradigma Funcional - Haskell



Comparativa Técnica entre Paradigmas.

La siguiente tabla resume las diferencias observadas al resolver el mismo problema en Prolog y en Haskell.

Aspecto	Prolog (lógico)	Haskell (funcional)
Representación de los vuelos	Hechos dinámicos vuelo/7 en la base de conocimientos	Registro Vuelo (ADT con record syntax) en una lista inmutable
Estado del sistema	Base de conocimientos modificable con assertz/1 y retractall/1	Grafo y TMC pasados como parámetros inmutables
Exploración de rutas	Backtracking implícito y resolución SLD	Recursión explícita y list comprehensions
Control de ciclos	Negación por fallo: \+ member/2	notElem sobre la lista de ciudades visitadas
Obtención de todas las soluciones	Meta-predicado findall/3	Lista de rutas construida de forma perezosa
Ordenamiento	quicksort/3 con call/N (criterio como predicado)	quicksortPor que ordena usando una función de clave (key function), no una función de comparación
Selección del óptimo	Búsqueda lineal del mínimo via mejor_ruta_costo/3 y mejor_ruta_tiempo/3	Recorrido lineal con foldl vía mejorRutaPor
Entrada/Salida	Predicados con efectos laterales (open/3, read_line_to_string/2)	Aislada en la mónada IO; el resto es puro
Detección de errores	En ejecución (tipado dinámico)	Muchos errores se detectan al compilar (tipado estático)

En **Prolog**, la lógica del problema se expresa de forma muy cercana a su enunciado: el motor se encarga de recorrer las alternativas y descartar las que violan alguna restricción. En **Haskell**, el recorrido debe escribirse explícitamente, pero el resultado es una función pura, fácil de probar de forma aislada y respaldada por el verificador de tipos. En ambos casos, la normalización de las horas a minutos enteros simplificó todas las validaciones de tiempo.

Conclusión

El desarrollo del sistema LogiExpress permitió contrastar de manera práctica dos de los paradigmas declarativos más potentes de la computación: el paradigma lógico con SWI-Prolog y el paradigma funcional con Haskell. Ambos enfoques demostraron una gran elegancia para abordar el problema clásico de búsqueda de rutas en grafos dirigidos, superando la complejidad del enfoque imperativo tradicional al centrarse en describir qué relaciones y propiedades deben cumplirse en lugar de gestionar estados mutables paso a paso.

En el motor de búsqueda, la imposición de restricciones operativas complejas como el Tiempo Mínimo de Conexión (TMC), la compatibilidad de días de vuelo, el límite de escalas y la prevención de ciclos se resolvió con éxito en ambos lenguajes. Prolog aprovechó de forma natural el mecanismo de unificación y la búsqueda en profundidad (DFS) con backtracking automático, mientras que Haskell ofreció una solución determinista y altamente confiable gracias a su sistema de tipos algebraicos estrictos, funciones de orden superior y evaluación pura sobre listas inmutables.

Finalmente, la integración de métricas de optimización mediante el algoritmo Quicksort y las funciones de selección de óptimos dotó al sistema de una capacidad analítica completa para identificar tanto las rutas más económicas como las más rápidas. Como se resume en la comparativa técnica, ambos paradigmas resultan herramientas complementarias para el modelado de redes de transporte y la toma de decisiones en el mundo real.